Complete Study Notes

#	Topic	Description
1	**What is Libuv?**	Introduction & Overview
2	**Architecture**	Core Components & Design
3	**Event Loop**	How the Loop Works (Step-by-Step)
4	**Handles & Requests**	I/O Primitives
5	**Thread Pool**	libuv's Worker Threads
6	**Async I/O Deep Dive**	Network, File, DNS
7	**Timers**	setTimeout / setInterval Internals
8	**Streams & Pipes**	TCP, UDP, Pipes
9	**Libuv in Node.js**	How Node.js uses Libuv
10	**Summary & Key Points**	Quick Revision

Chapter 1 — What is Libuv?

Definition

Libuv is a **multi-platform C library** that provides support for **asynchronous I/O** (input/output) and other features needed for building high-performance, non-blocking applications. It was originally created for **Node.js** but is now used in many other projects.

> ■ **Simple Analogy**
>
> Imagine you are a chef (your program) in a restaurant. Instead of cooking one dish and waiting for it to finish before starting the next, you **start multiple dishes at once** and get notified when each is ready. Libuv is the **kitchen management system** that lets you do this efficiently.

Key Characteristics

Feature	What it means
■ **Cross-Platform**	Works on Linux, macOS, Windows — same API everywhere.
■ **Event-Driven**	Uses an event loop to react to events (data received, file read, timer fired).
■ **Non-Blocking I/O**	Your program doesn't pause while waiting for slow operations.
■ **Written in C**	Very fast and low-level. Higher-level languages (Node.js) wrap it.
■ **Open Source**	MIT licensed, actively maintained on GitHub.

Why Was Libuv Created?

Before Libuv, writing async code that worked on **both Windows and Linux** was painful. Each OS has a different mechanism: Linux uses **epoll**, macOS uses **kqueue**, and Windows uses **IOCP (I/O Completion Ports)**. Libuv **hides all this complexity** behind a single, unified API so developers don't have to worry about OS differences.

Chapter 2 — Architecture & Core Components

High-Level Architecture Diagram

YOUR APPLICATION (Node.js / C program)		
▼ calls into ▼		
LIBUV LIBRARY		
■ **Event Loop** Central scheduler. Checks for pending events and dispatches callbacks.	■ **Thread Pool** Worker threads for blocking ops like file I/O and DNS.	■ **I/O Polling** OS-level (epoll/kqueue/IOCP) to monitor sockets and file descriptors.
■ **Timers** setTimeout, setInterval — scheduled callbacks.	■ **Handles** Long-lived objects: TCP server, UDP socket, file watcher.	■ **Requests** One-shot operations: DNS lookup, file read, write.

OS-Level Interfaces Used by Libuv

Operating System	Mechanism Used	What It Does
Linux	`epoll`	Efficiently watches many file descriptors for events
macOS / BSD	`kqueue`	Kernel event queue — similar purpose to epoll
Windows	`IOCP`	I/O Completion Ports — Windows native async model
Solaris	`event ports`	Similar to kqueue for Solaris OS

Chapter 3 — The Event Loop (Heart of Libuv)

What is the Event Loop?

The **Event Loop** is a loop that keeps running as long as there is work to do. It continuously checks: *"Are there any events or tasks ready?"* and if yes, it runs the associated **callback function**. This is what makes Node.js single-threaded yet able to handle thousands of concurrent operations.

■ Key Insight

The event loop does NOT run tasks in parallel on the main thread. Instead, it **delegates** slow operations (like reading a file) to the OS or worker threads and moves on. When the result is ready, the callback is queued and executed later.

Phases of the Event Loop (Step-by-Step)

Each iteration of the event loop is called a **tick**. Every tick goes through these phases in order:

Phase 1	**Timers**	Run callbacks scheduled by setTimeout() and setInterval() whose delay has expired.
Phase 2	**Pending I/O Callbacks**	Run I/O callbacks that were deferred to the next loop iteration (errors from previous ops).
Phase 3	**Idle / Prepare**	Internal use only — libuv uses this phase for internal housekeeping. You rarely interact here.
Phase 4	**I/O Poll**	This is the MOST IMPORTANT phase. Libuv asks the OS: 'Any I/O events ready?' If yes, run those callbacks. If no, wait (block) for new events up to a calculated timeout.
Phase 5	**Check**	Run callbacks scheduled with setImmediate() — these always run after I/O polling.
Phase 6	**Close Callbacks**	Run close event callbacks, e.g., when a socket or handle is closed (socket.on('close', ...)).

■ Between Every Phase: microtasks!

Before moving to the next phase, Node.js drains the **microtask queue**:

- **process.nextTick()** callbacks run first (highest priority).
- **Promise.then()** / **queueMicrotask()** callbacks run next.

Microtasks always run before the loop moves to the next phase.

Event Loop — Simplified Flow Diagram

START	uv_run() called
→	Any timers expired? → Run timer callbacks
→	Any pending I/O callbacks? → Run them
→	I/O POLL: Ask OS for events → Run I/O callbacks
→	Run setImmediate() callbacks (Check phase)
→	Run close callbacks
■	Any handles/requests alive? → LOOP AGAIN
STOP	No work left → uv_run() returns

Chapter 4 — Handles & Requests

■ HANDLES (Long-Lived)

- Represent long-lived objects
- Active as long as the loop runs
- Examples: TCP server, UDP socket, timer, file watcher
- Use: uv_tcp_t, uv_udp_t, uv_timer_t, uv_fs_event_t
- Reference counted — loop stays alive if handle is active

■ REQUESTS (One-Shot)

- Represent one-shot (single) operations
- Completed and discarded after callback
- Examples: file read, DNS lookup, TCP connect
- Use: uv_fs_t, uv_getaddrinfo_t, uv_connect_t
- Don't keep the loop alive on their own

Common Handle Types

Handle	Type Name	Use Case
TCP Socket	`uv_tcp_t`	Client/server TCP connections
UDP Socket	`uv_udp_t`	Datagram (UDP) communication
Timer	`uv_timer_t`	setTimeout / setInterval behaviour
File Event	`uv_fs_event_t`	Watch file/directory for changes
Process	`uv_process_t`	Spawn and monitor child processes
Idle	`uv_idle_t`	Run callback every loop tick when idle
Signal	`uv_signal_t`	Handle OS signals like SIGINT, SIGTERM
Pipe	`uv_pipe_t`	IPC pipes or named pipes

Chapter 5 — The Thread Pool

Why Does Libuv Need a Thread Pool?

Some operations **cannot be made truly async** at the OS level — for example, reading local files using standard POSIX calls on Linux. These calls **block** the calling thread until the data is ready. If libuv did this on the main thread, the entire event loop would freeze!

To solve this, libuv maintains a **pool of background worker threads** that handle these blocking operations. The main thread stays free and is notified via a callback when the background work is done.

■ Thread Pool Default Size

By default, libuv creates **4 worker threads** in the pool.

You can change this by setting the environment variable:

```
UV_THREADPOOL_SIZE=8 (max recommended: 128)
```

More threads can help if you have many concurrent file I/O operations.

What Uses the Thread Pool?

File System Operations	fs.readFile(), fs.writeFile(), fs.stat() — any file I/O
DNS Resolution	dns.lookup() — resolves domain names to IPs
Crypto Operations	crypto.pbkdf2(), crypto.randomBytes() — expensive hashing
User C++ Addons	Any native Node.js addon that uses libuv's work queue API

How Thread Pool Works — Step by Step

Step 1	Main thread calls fs.readFile('data.txt', callback)
Step 2	Libuv receives the request and puts it in the work queue
Step 3	A free worker thread picks up the task and reads the file (blocking call)
Step 4	Worker thread completes and puts the result in the done queue
Step 5	Event loop picks up the completed result during the I/O poll phase
Step 6	Your callback is called with the file data — back on the main thread!

Chapter 6 — Async I/O Deep Dive

Network I/O (Sockets)

Network operations like TCP/UDP are **truly async at the OS level**. Libuv registers interest in socket events with the OS using epoll/kqueue/IOCP. The OS notifies libuv when data arrives or a connection is established. **No worker thread needed!**

File I/O

File I/O is different. Most OS file APIs are **blocking**. So libuv offloads them to the **thread pool**. On Linux, there is a newer API called **io_uring** that makes file I/O truly async — libuv has support for this too in newer versions.

DNS Resolution

DNS lookup (e.g., resolving 'google.com' to an IP) uses the system's **getaddrinfo()** call which is **blocking**. Libuv runs this in the thread pool and notifies your callback when done.

I/O Type	Blocking?	How Libuv Handles
TCP/UDP Network	No (async at OS)	OS (epoll/kqueue/IOCP) — no thread pool
File Read/Write	Yes (blocking)	Thread pool — worker thread runs it
DNS Lookup	Yes (getaddrinfo)	Thread pool
Pipes (IPC)	No (mostly async)	OS-level I/O polling
Child Process	No	OS signals + I/O polling

Chapter 7 — Timers in Libuv

How Timers Work Internally

When you call **setTimeout(fn, 100)** in Node.js, libuv internally creates a **uv_timer_t** handle set to fire after 100ms. Libuv maintains a **min-heap** (sorted by expiry time) of all pending timers. At the start of each event loop tick, libuv checks: *Has any timer's deadline passed?* If yes, the callback is executed.

■■ Important: Timers Are NOT Precise

setTimeout(fn, 100) means **at LEAST 100ms**, not exactly 100ms. If the event loop is busy processing I/O callbacks, your timer callback may fire late. This is expected behavior.

Function	Libuv API	Behaviour
`setTimeout(fn, ms)`	`uv_timer_start() once`	Fires callback once after ms milliseconds
`setInterval(fn, ms)`	`uv_timer_start() repeat`	Fires callback every ms milliseconds
`clearTimeout(id)`	`uv_timer_stop()`	Cancels the pending timer
`setImmediate(fn)`	`Check phase callback`	Fires after current I/O phase, before timers
`process.nextTick(fn)`	`Microtask queue`	Fires before ANYTHING else in next phase

Chapter 8 — Streams, TCP & Pipes

Streams in Libuv

A **stream** in libuv is an abstraction over a readable/writable connection — like a TCP socket, a pipe, or a TTY (terminal). Streams support **reading data in chunks** (non-blocking) and **writing data with backpressure handling**.

Type	Name	Used For
`uv_tcp_t`	**TCP stream**	Client and server TCP connections over the network
`uv_pipe_t`	**Pipe stream**	IPC (inter-process communication) or named pipes
`uv_tty_t`	**TTY stream**	Terminal input/output (stdin, stdout, stderr)
`uv_udp_t`	**UDP socket**	Connectionless UDP datagrams (not a stream, but similar API)

TCP Server Lifecycle with Libuv

1. `uv_tcp_init(loop, &server;)` — Initialize the TCP handle
2. `uv_ip4_addr('0.0.0.0', 8080, &addr;)` — Set up the address
3. `uv_tcp_bind(&server;, &addr;, 0)` — Bind to port 8080
4. `uv_listen(&server;, 128, on_connect_cb)` — Start listening; callback fires on new client
5. `uv_tcp_init(loop, &client;); uv_accept(&server;, &client;)` — Accept the client
6. `uv_read_start(&client;, alloc_cb, read_cb)` — Start reading data from client
7. `uv_write(&req;, &client;, buf, 1, write_cb)` — Send response
8. `uv_close(&client;, close_cb)` — Close the connection when done

Chapter 9 — How Node.js Uses Libuv

Node.js Architecture Overview

Node.js is built on top of two major components: **V8** (Google's JavaScript engine that executes JS code) and **Libuv** (which handles all async I/O). Here's how they connect:

Layer	Component	Responsibility
Topmost	**Your JS Code**	Business logic written by you
JS Engine	**V8**	Parses & executes JavaScript
Bindings	**Node.js C++ Bindings**	Bridge between V8 and libuv (N-API / libuv calls)
Async Runtime	**Libuv**	Event loop, thread pool, I/O, timers
OS Layer	**Operating System**	epoll / kqueue / IOCP / kernel

End-to-End Example: fs.readFile()

Let's trace exactly what happens when you write:

```
fs.readFile('hello.txt', (err, data) => { console.log(data); });
```

JS Layer	→ Node.js calls the C++ binding for fs.readFile
C++ Binding	→ Creates a uv_fs_t request and calls uv_fs_read()
Libuv Queue	→ Request is placed in the thread pool work queue
Worker Thread	→ A background thread runs open()+read() — blocking C calls
Done Queue	→ Thread marks the request done with the file data
Event Loop	→ Next I/O poll phase — libuv finds the done request
Callback	→ JS callback is scheduled and called with (null, data)
Your Code	→ console.log(data) runs — you see the file content!

Chapter 10 — Summary & Quick Revision

Key Points to Remember

✔ What is Libuv?	A C library providing async I/O, event loop, and cross-platform support. Used by Node.js.
✔ Event Loop	Runs in phases: Timers → Pending I/O → Idle/Prepare → Poll → Check → Close. Repeats till no work left.
✔ Poll Phase	Most important phase — OS is asked for I/O events. Loop blocks here if nothing else is pending.
✔ Thread Pool	Default 4 threads. Used for file I/O, DNS, crypto. Configured via UV_THREADPOOL_SIZE.
✔ Network I/O	Truly async — done via OS (epoll/kqueue/IOCP). No thread pool needed.
✔ File I/O	Blocking calls offloaded to thread pool. Worker thread reads file; callback fires via event loop.
✔ Handles	Long-lived libuv objects (TCP server, timer). Keep the loop alive.
✔ Requests	One-shot operations (file read, DNS). Don't keep loop alive.
✔ Microtasks	process.nextTick() and Promise.then() run BETWEEN every phase of the loop.
✔ setImmediate vs setTimeout(0)	setImmediate fires in Check phase (after I/O). setTimeout fires in Timer phase (may vary in order).